

Abstract

This project describes the design and implementation of a Secure Process Resource Monitoring and Limiting System for Linux. The primary motivation for this system is to detect and prevent the abuse of any process resource, which may affect the performance and stability of the computer system.

The proposed system has been designed by developing three major modules: process monitoring, suspicious behavior detection, and response control.

The Monitoring module collects dynamic information regarding the processes being executed on the system at a specific time, including the percentage of CPU utilization, memory usage, the number of child processes, and the number of open file descriptors. The Detection module evaluates this collected data to detect suspicious activities based on predetermined limits, which include high CPU usage, high memory usage, or possible fork bomb activity. When suspicious activity is found, alerts are generated and then delivered to the Response module.

The Response module takes care of all the generated alerts using a step-by-step approach. It initially logs the event, then in case of recurrence of the suspicious behavior of the same process, it lowers its priority level and further restricts the execution of the same in the worst case scenario. Such an approach ensures the practicality of the system as there is no abrupt termination of a process from the very beginning.

The entire process demonstrates the use of monitoring, detection, and control techniques in ensuring system security. It also helped us understand Linux process management, teamwork, and how simple security mechanisms can be used to protect system resources.

Symbols, Keywords, and Abbreviations

CPU — Central Processing Unit
RAM — Random Access Memory
PID — Process Identifier
OS — Operating System
DoS — Denial of Service
JSON — JavaScript Object Notation
PCB — Process Control Block
cgroup — Control Group
SJF — Shortest Job First
MMU — Memory Management Unit

Introduction

1.1 Background

Modern operating systems are designed to support multiple processes running simultaneously while sharing system resources like memory and CPU time. Efficient resource management is essential to ensure system stability, performance, and fairness among processes. However, some processes may consume excessive resources, this can be due to programming errors, inefficient design or malicious intent.

A common issue in such environments is resource exhaustion, where one (or more) processes consume disproportionate amount of CPU or memory, which then negatively affects the performance of other processes. This can lead to system slowdown, application crashes or complete system failure.

This problem is closely related to the concept of Denial Of Service (DoS), an attack where system resources are overwhelmed, which makes services unavailable to legitimate users. Dos can occur internally within a system when a process monopolizes resources.

1.2 Importance of process monitoring and control

Hence, monitoring and controlled process behavior is critical for the security and reliability of operating systems. Without proper monitoring, a single faulty process can degrade the performance of the entire system. This is especially critical in multi-user or multi-process environments where fairness and availability are essential.

Process monitoring and control are resource control used often in real world systems.

Operating systems like Linux manage priorities to ensure fair CPU allocation. Technologies such as cgroup are used to limit and isolate resource usage.

1.3 Overview

The objective of this project is to develop a tool that monitors running processes and prevents resource abuse within a system. The tool tracks CPU and memory usage, detects abnormal behavior, and applies control mechanisms such as reducing process priority or isolating the process.

The project focuses on 3 main features: process behavior monitoring, process priority control, and automatic process isolation, to enhance system security and reliability.

Design Methodology

1.1 Overview

The project follows a modular design approach, which means that the system is made up of three main parts that work together to detect and respond to suspicious processes. Each module has a specific responsibility, and all modules are connected through data files.

This design makes the system:

- Easy to understand
- Easy to test
- Easy to debug

1.2 System Architecture

- **Process Monitoring**
Collects real-time data about all running processes such as CPU usage, memory usage, number of child processes, and open files.
- **Behavior Detection**
Analyzes the collected data and checks if any process exceeds predefined thresholds. If suspicious behavior is detected, an alert is generated.
- **Response Controller**
Receives alerts and takes action using a step-by-step escalation strategy (log → reduce priority → terminate process).

1.3 Data Flow Design

The modules communicate using JSON files:

- **process_data.json**
 - Created by the monitoring module
 - Contains all process information
- **alerts.json**
 - Created by the detection module
 - Contains suspicious processes

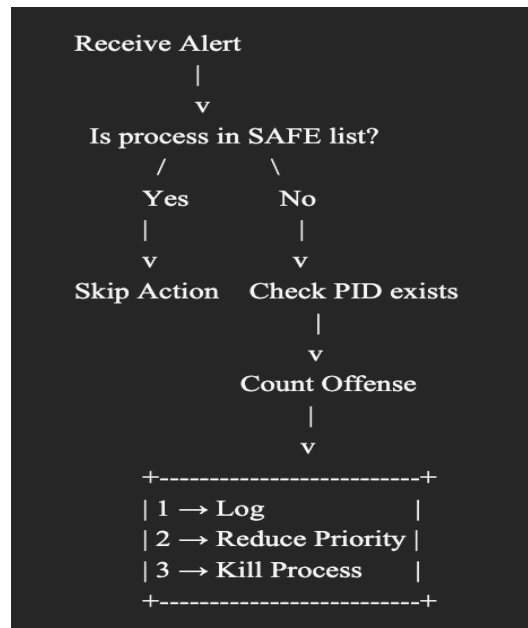
This file-based communication ensures:

- loose coupling between modules
- easier debugging
- independent testing of each module

1.4 Flow of Execution

The system runs continuously in a loop:

1. The monitoring module scans all processes
2. It saves process data into process_data.json
3. The detection module reads this file
4. It checks for abnormal behavior based on thresholds
5. Suspicious processes are written into alerts.json
6. The response module reads the alerts
7. It takes action based on the offense level



1.5 Response Strategy Design

The response module uses a gradual escalation approach:

- *First offense:* Log the suspicious activity
- *Second offense:* Reduce process priority using renice
- *Third offense:* Terminate the process

This design ensures:

- Safe handling of processes
- Avoids unnecessary termination
- Mimics real-world security systems

1.6 Safety Mechanism

To prevent system instability, a safe process list is implemented.

Protected processes include:

- System services (e.g., systemd, Xorg)
terminal processes (e.g., bash, qterminal)

These processes are ignored during response actions.

1.7 Design Justification

This design was chosen because:

- It separates responsibilities clearly
- It allows teamwork and integration
- It is scalable and easy to improve
- It follows real-world security architecture (monitor → detect → respond)

Implementation Details

4.1 Overview

4.1.1 process monitoring module

Process Monitoring Module has been developed in order to capture real-time information of all processes running in the system. This module will constantly scan the system and gather useful information like process ID, process name, CPU utilization, memory utilization, number of child processes and number of open files.

This module is programmed in Python 3 with help of psutil library which gives access to system-level process information. All the collected information is displayed on the terminal and will also be saved in a JSON formatted file (process_data.json).

The monitoring runs in a continuous loop with fixed time interval, to ensure that process information is updated in real time.

4.1.2 Experimental setup and tools used

This module was implemented and tested in kali linux environment using python3. The main library used was psutil for retrieving process information such as CPU usage, memory usage, child processes and open files

Other libraries used:

- json for storing process data in structured format
- time for implementing periodic monitoring

During testing this module was run in one terminal window, continuously updating process data, while other scripts were executed in separate terminals to simulate different process behaviors.

4.1.3 verification of results

The output results were verified by observing both the terminal output and the generated process_data.json file. The module was tested under normal and abnormal conditions.

In normal execution, the module successfully displayed all running processes with correct CPU and memory values. In testing scenarios, additional scripts were used to simulate high CPU usage, multiple child processes, and increased file usage. The monitoring module correctly reflected these changes in real time.

The JSON output was also checked to ensure that all process information was accurately recorded and could be used by the detection module.

Output sample:

Running the monitoring module monitor.py on the terminal:

```
reem@kali: ~  
Session Actions Edit View Help  
(reem@kali)-[~]  
$ sudo python3 monitor.py  
[sudo] password for reem:  
[START] Process Monitoring Module Running...  
  
[INFO] Monitoring running processes ...  
  
PID      Name                CPU %    Memory(MB)    Children    Files  
1        systemd             0.00     15.60         110         4  
2        kthreadd             0.00     0.00          83         0  
3        pool_workqueue_release0.00    0.00         0         0  
4        kworker/R-rcu_gp     0.00     0.00          0         0  
5        kworker/R-sync_wq    0.00     0.00          0         0  
6        kworker/R-kvfree_rcu_reclaim0.00 0.00         0         0  
7        kworker/R-slub_flushwq0.00 0.00         0         0  
8        kworker/R-netns      0.00     0.00          0         0  
10       kworker/0:0H-events_highpri0.00 0.00         0         0  
13       kworker/R-mm_percpu_wq0.00 0.00         0         0  
14       ksoftirqd/0         0.00     0.00          0         0  
15       rcu_preempt         0.00     0.00          0         0  
16       rcu_exp_par_gp_kthread_worker0.00 0.00         0         0  
17       rcu_exp_gp_kthread_worker0.00 0.00         0         0  
18       migration/0         0.00     0.00          0         0  
19       idle_inject/0       0.00     0.00          0         0  
20       cpuhp/0             0.00     0.00          0         0  
21       cpuhp/1             0.00     0.00          0         0  
22       idle_inject/1       0.00     0.00          0         0  
23       migration/1         0.00     0.00          0         0  
24       ksoftirqd/1         0.00     0.00          0         0  
26       kworker/1:0H-kblockd0.00 0.00         0         0  
27       kworker/u9:0-events_unbound0.00 0.00         0         0  
31       kdevtmpfs           0.00     0.00          0         0  
32       kworker/R-inet_frag_wq0.00 0.00         0         0  
33       rcu_tasks_kthread   0.00     0.00          0         0  
34       rcu_tasks_rude_kthread0.00 0.00         0         0  
35       rcu_tasks_trace_kthread0.00 0.00         0         0  
36       kauditd             0.00     0.00          0         0  
37       khungtaskd          0.00     0.00          0         0  
38       oom_reaper          0.00     0.00          0         0  
39       kworker/u9:1-kvfree_rcu_reclaim0.00 0.00         0         0  
40       kworker/R-writeback 0.00     0.00          0         0  
42       kcompactd0          0.00     0.00          0         0  
43       ksmd                0.00     0.00          0         0  
44       khugepaged          0.00     0.00          0         0  
45       kworker/R-kblockd   0.00     0.00          0         0  
46       kworker/R-blkcg_punt_bio0.00 0.00         0         0  
47       kworker/R-kintegrityd0.00 0.00         0         0  
48       irq/9-acpi          0.00     0.00          0         0
```

We can use this data to see which processes are using up the most CPU such as:

195706	sudo	0.00	2.34	1	1
195707	python3	99.30	14.09	0	0

Here we can see that the python3 process with our code is using 99.3% of CPU

Process_data.json file:

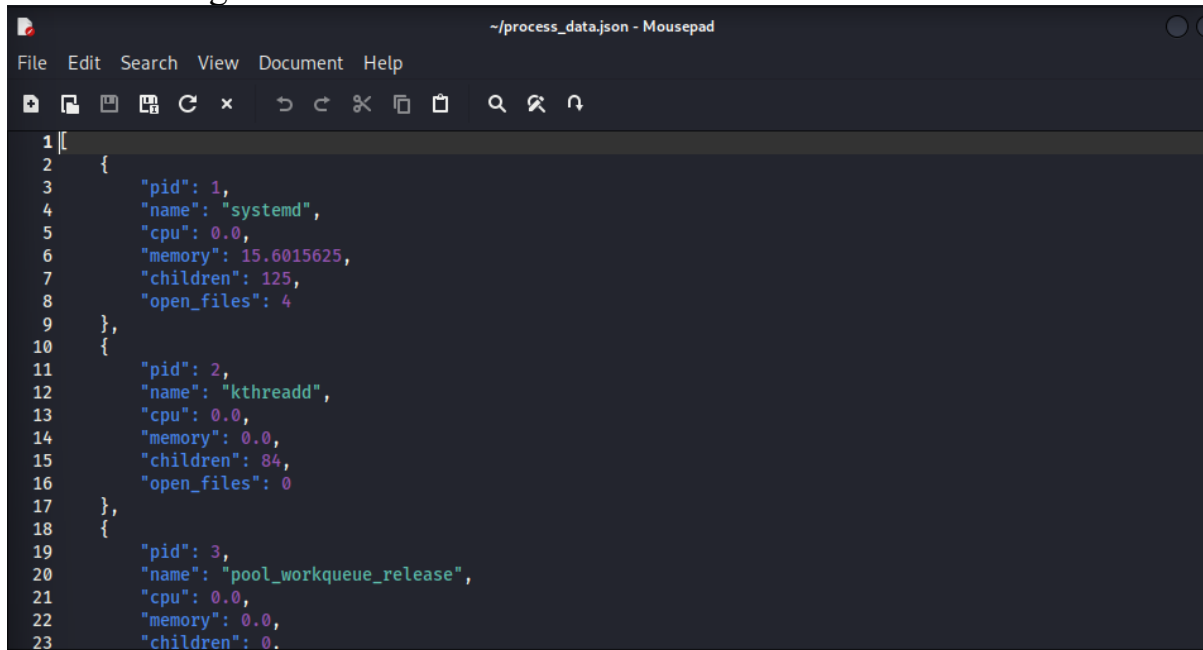
This is a file our code creates automatically

`export_to_json(processes)` line calls the function

With `open(filename, "w")` as `f`:

`Json.dump(processes, f, indent=4)`

So a file called `process_data.json` is created and it contains all the process data the monitoring module collects

A screenshot of a text editor window titled "-/process_data.json - Mousepad". The editor shows a JSON array of process data. The first process is 'systemd' with pid 1, 0.0% CPU, 15.6015625 memory, 125 children, and 4 open files. The second process is 'kthreadd' with pid 2, 0.0% CPU, 0.0 memory, 84 children, and 0 open files. The third process is 'pool_workqueue_release' with pid 3, 0.0% CPU, 0.0 memory, and 0 children. The JSON is formatted with 4-space indentation.

```
1[
2  {
3      "pid": 1,
4      "name": "systemd",
5      "cpu": 0.0,
6      "memory": 15.6015625,
7      "children": 125,
8      "open_files": 4
9  },
10 {
11     "pid": 2,
12     "name": "kthreadd",
13     "cpu": 0.0,
14     "memory": 0.0,
15     "children": 84,
16     "open_files": 0
17 },
18 {
19     "pid": 3,
20     "name": "pool_workqueue_release",
21     "cpu": 0.0,
22     "memory": 0.0,
23     "children": 0.
```

4.2 Overview

4.2.1 Suspicious Behavior Detection Module

The Suspicious Behavior Detection Module was developed to analyze the process data captured by the monitoring module. It uses process_data.json file, which stores data regarding running processes including process ID, process name, CPU, memory usage, number of child processes and number of opened files. It was programmed with Python 3 language and was intended to perform continuous analysis at regular time intervals.

Suspicious behavior was detected through comparison of the received data to predefined thresholds. The program was able to flag processes for suspicious behavior according to how much CPU, memory they use, how many child processes and opened files they have. In order to limit number of false positives, a list of trusted processes is also available so that certain system processes will not be flagged unnecessary.

4.2.2 Experimental Setup and Tools Used

This module testing was performed using Kali Linux operating system with Python 3. Required modules were json (file manipulation), and time (repeated tasks). During the test, the monitoring module was executed in one console window, and then the detection module was run separately in another console window. It ensured that process data was checked regularly with the latest updates.

Module results were recorded in two outputs. Suspicious processes were saved into alerts.json for further analysis by the response module, while detections were also displayed in console.

4.2.3 Verification of Results

Results were checked by verifying if alert was triggered when processes violated their limits. Validation was made based on the console output and content of alerts.json file. For the normal condition, the module showed that no suspicious behavior was observed. As for abnormal conditions, high CPU or memory usage resulted in triggering an alert containing proper process ID, process name, and type of violation.

Read process data → compare with limits → ignore trusted processes when needed → generate alert → save output

Output sample:

Without the monitoring module running:

```
[START] Suspicious Behavior Detection Module Running ...  
[ERROR] process_data.json not found. Make sure the process monitoring module is running.
```

- When the monitoring module was not running, the detection module showed an error because it could not find `process_data.json`.

After the monitoring module started running:

The left terminal window shows a table of process data with columns: Session, Actions, Edit, View, Help. The data is as follows:

Session	Actions	Edit	View	Help
0				
74	kworker/R-ipv6_addrconf0.00	0.00	0	
79	kworker/R-kstrp	0.00	0.00	0
85	kworker/u17:0	0.00	0.00	0
156	kworker/1:1H-kblockd0.00	0.00	0	
188	kworker/2:1H-kblockd0.00	0.00	0	
191	kworker/3:1H-kblockd0.00	0.00	0	
281	kworker/1:2-events	0.00	0.00	0
286	jbd2/vda3-8	0.00	0.00	0
287	kworker/R-ext4-rsv-conversion0.00	0.00		
311	hwng	0.00	0.00	0
352	systemd-journald	0.00	15.36	0
380	systemd-timesyncd	0.00	7.08	0
386	systemd-udev	0.00	11.53	0
394	psimon	0.00	0.00	0
463	scsi_eh_0	0.00	0.00	0
464	kworker/R-scsi_tmf_00.00	0.00	0	

The right terminal window shows the output of the detection module. It displays a list of alerts and data sent to the response module. The alerts are as follows:

```
{'pid': 9551, 'name': 'Isolated Web Co', 'violation': 'CPU Abuse'}  
{'pid': 9551, 'name': 'Isolated Web Co', 'violation': 'Memory Abuse'}  
{'pid': 10257, 'name': 'python3', 'violation': 'CPU Abuse'}  
  
[2026-04-02 21:58:13] ALERT: PID 1223 (xfce4-panel) possible fork bomb  
[2026-04-02 21:58:13] ALERT: PID 9282 (firefox-esr) exceeds memory limit  
[2026-04-02 21:58:13] ALERT: PID 9282 (firefox-esr) too many open files  
[2026-04-02 21:58:13] ALERT: PID 9551 (Isolated Web Co) exceeds CPU limit  
[2026-04-02 21:58:13] ALERT: PID 9551 (Isolated Web Co) exceeds memory limit  
  
Data to send to response module:  
{'pid': 1223, 'name': 'xfce4-panel', 'violation': 'Fork Bomb'}  
{'pid': 9282, 'name': 'firefox-esr', 'violation': 'Memory Abuse'}  
{'pid': 9282, 'name': 'firefox-esr', 'violation': 'File Descriptor Abuse'}  
{'pid': 9551, 'name': 'Isolated Web Co', 'violation': 'CPU Abuse'}  
{'pid': 9551, 'name': 'Isolated Web Co', 'violation': 'Memory Abuse'}  
  
[2026-04-02 21:58:15] ALERT: PID 1223 (xfce4-panel) possible f
```

- After the monitoring module started, the detection module successfully read live process data, detected suspicious resource usage, and generated alerts containing the process ID, process name, and type of violation. It also applied the threshold rules. It detected possible fork bomb behavior in `xfce4-panel`, high memory and open file usage in `firefox-esr`, and high CPU and memory usage in `Isolated Web Co`. The generated alerts were then prepared and sent to the response module.

4.3.1 Overview

The response module is responsible for taking action on suspicious processes detected by alerts. It reads alerts from alerts.json and applies appropriate responses based on the severity and repetition of the violation.

4.3.2 Experimental Setup and Tools Used

This module was tested in a Kali Linux environment using Python 3. The main libraries used were os, json, time, signal, subprocess, and datetime.

During testing, the monitoring module was run in one terminal window, the detection module in another, and the response module in a third terminal window. This allowed the system to operate in real time.

The response module was executed using:

```
sudo python3 response.py
```

This was required to allow actions such as reducing process priority and terminating processes.

The output of this module was displayed in the terminal using colored messages and also saved in a log file (response_logs.txt).

4.3.3 Verification of Results

The output results were verified by checking whether the correct actions were applied to suspicious processes. Verification was done by observing the terminal output and checking the log file.

In normal cases, when no suspicious processes were detected, the module continued running without taking action. When alerts were received, the module followed the correct sequence of actions: logging the alert, reducing priority, and terminating the process if the behavior continued.

The results were also verified by observing system behavior. When a process was terminated, the corresponding application closed, confirming that the kill action was successful. Protected processes were also tested and confirmed to be skipped, ensuring that the safety mechanism worked correctly.

Output Sample:

When the detection module was not running, the response module did not receive any alerts and remained idle without taking any action.

```
(aseel@aseel)-[~/Desktop/secureos]
$ python3 response.py
[2026-04-02 22:46:04] Response module started
```

Figure 1: Without detection module running

After the detection module started, the response module successfully received alerts, processed suspicious processes, and applied actions based on the escalation strategy. The output showed the process ID, process name, and the action taken (log, priority reduction, or termination).

```
[2026-04-02 22:49:26] PID 260528: first offense → logging only
[2026-04-02 22:49:26] ALERT: PID 260559 (WebExtensions) detected for Memory Abuse - offense #1
[2026-04-02 22:49:26] PID 260559: first offense → logging only
[2026-04-02 22:49:28] PID 663 (Xorg) is protected by name. Skipping action.
[2026-04-02 22:49:28] PID 205597 (xfce4-panel) is protected by name. Skipping action.
[2026-04-02 22:49:28] PID 205725 (xfwm4) is protected by name. Skipping action.
[2026-04-02 22:49:28] ALERT: PID 260405 (firefox-esr) detected for Memory Abuse - offense #3
[2026-04-02 22:49:28] PID 260405: repeated offense → killing process
[2026-04-02 22:49:28] PID 260405: process killed
[2026-04-02 22:49:28] ALERT: PID 260405 (firefox-esr) detected for File Descriptor Abuse - offense #1
[2026-04-02 22:49:28] PID 260405: first offense → logging only
[2026-04-02 22:49:28] ALERT: PID 260477 (Privileged Cont) detected for Memory Abuse - offense #2
[2026-04-02 22:49:28] PID 260477: second offense → reducing priority
260477 (process ID) old priority 0, new priority 10
[2026-04-02 22:49:28] PID 260477: priority reduced
[2026-04-02 22:49:28] ALERT: PID 260528 (file:// Content) detected for Memory Abuse - offense #2
[2026-04-02 22:49:28] PID 260528: second offense → reducing priority
260528 (process ID) old priority 0, new priority 10
[2026-04-02 22:49:28] PID 260528: priority reduced
[2026-04-02 22:49:29] ALERT: PID 260559 (WebExtensions) detected for Memory Abuse - offense #2
```

Figure 2: After detection module started running

4.3.5 Colored Output Explanation

To improve readability and make the system output clearer, colored messages were implemented in the response module. Each color represents a different type of action or status:

- **Blue** - Informational messages
Used for general system updates such as module start, skipped actions, or safe process notifications.
- **Orange** - Alerts
Indicates that a suspicious process has been detected by the system.
- **Green** - Successful actions
Shows that a corrective action was successfully applied, such as reducing process priority.
- **Red** - Critical actions or errors
Represents important actions such as process termination or failure messages.

Implementation Code and Results

monitor.py

```
import psutil
import time
import json

# Time between each monitoring round
REFRESH_INTERVAL = 2 # seconds

def get_process_info(proc):
    try:
        pid = proc.pid
        name = proc.name()

        # Get current CPU usage
        cpu = proc.cpu_percent(interval=None)

        # Convert memory from bytes to MB
        memory = proc.memory_info().rss / (1024 * 1024)

        # Count child processes
        children = len(proc.children(recursive=True))

        # Count open files
        try:
            open_files = len(proc.open_files())
        except Exception:
            open_files = 0

        return {
            "pid": pid,
            "name": name,
            "cpu": cpu,
            "memory": memory,
            "children": children,
            "open_files": open_files
        }

    except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
        return None

def monitor_processes():
    process_data = []

    # Loop through all running processes
    for proc in psutil.process_iter():
        info = get_process_info(proc)
        if info:
```

```

        process_data.append(info)

    return process_data

def display_processes(processes):
    print("\n[INFO] Monitoring running processes...\n")
    print(f'{"PID":<8}{"Name":<20}{"CPU %":<10}{"Memory(MB)":<15}{"Children":<10}{"Files":<10}')
```

PID	Name	CPU %	Memory(MB)	Children	Files
1	python3	0.0	10.2	1	10

```

    # Show process details in table form
    for p in processes:
        print(f'{"pid":<8}{"name":<20}{"cpu":<10.2f}{"memory":<15.2f}{"children":<10}{"open_files":<10}')
```

```

def export_to_json(processes, filename="process_data.json"):
    # Save process data for the detection module
    with open(filename, "w") as f:
        json.dump(processes, f, indent=4)

if __name__ == "__main__":
    print("[START] Process Monitoring Module Running...\n")

    # First CPU call is used to initialize readings
    for proc in psutil.process_iter():
        try:
            proc.cpu_percent(interval=None)
        except:
            pass

    while True:
        processes = monitor_processes()
        display_processes(processes)
        export_to_json(processes)
        time.sleep(REFRESH_INTERVAL)

```

alerts.py

```

import time
import json

```



```

# Time between each detection round
REFRESH_INTERVAL = 2

# Limits used to detect suspicious behavior
CPU_LIMIT = 50
MEMORY_LIMIT = 400
CHILDREN_LIMIT = 20
FILES_LIMIT = 50

# Trusted processes that should not be flagged easily
trusted_processes = [
    "systemd",
    "kthreadd",
    "lightdm",
    "xfce4-session",
    "Xorg"
]

LOG_FILE = "detection_logs.txt"

def write_log(message):
    # Add time to each log message
    current_time = time.strftime("%Y-%m-%d %H:%M:%S")
    full_message = f"[{current_time}] {message}"

    print(full_message)

    # Save the same message in a log file
    with open(LOG_FILE, "a") as file:
        file.write(full_message + "\n")

def load_process_data(filename="process_data.json"):
    try:
        # Read process data created by the monitoring module
        with open(filename, "r") as f:
            return json.load(f)

    except FileNotFoundError:
        print("[ERROR] process_data.json not found. Make sure the process monitoring module is running.\n")
        return None

    except json.JSONDecodeError:
        print("[ERROR] Could not read process_data.json right now. Trying again...\n")
        return None

def detect_suspicious_behavior(processes):
    alerts = []

```

```

# Check each process against the limits
for p in processes:
    pid = p["pid"]
    name = p["name"]
    cpu = p["cpu"]
    memory = p["memory"]
    children = p["children"]
    open_files = p["open_files"]

    is_trusted = name in trusted_processes

    if cpu > CPU_LIMIT and not is_trusted:
        message = f"ALERT: PID {pid} ({name}) exceeds CPU limit"
        write_log(message)
        alerts.append({
            "pid": pid,
            "name": name,
            "violation": "CPU Abuse"
        })

    if memory > MEMORY_LIMIT and not is_trusted:
        message = f"ALERT: PID {pid} ({name}) exceeds memory limit"
        write_log(message)
        alerts.append({
            "pid": pid,
            "name": name,
            "violation": "Memory Abuse"
        })

    if children > CHILDREN_LIMIT and not is_trusted:
        message = f"ALERT: PID {pid} ({name}) possible fork bomb"
        write_log(message)
        alerts.append({
            "pid": pid,
            "name": name,
            "violation": "Fork Bomb"
        })

    if open_files > FILES_LIMIT and not is_trusted:
        message = f"ALERT: PID {pid} ({name}) too many open files"
        write_log(message)
        alerts.append({
            "pid": pid,
            "name": name,
            "violation": "File Descriptor Abuse"
        })

    return alerts

def export_alerts(alerts, filename="alerts.json"):

```

```

# Save alerts for the response module
with open(filename, "w") as f:
    json.dump(alerts, f, indent=4)

def display_alerts(alerts):
    print("\nData to send to response module:")

    if len(alerts) == 0:
        print("No suspicious behavior detected.")
    else:
        for alert in alerts:
            print(alert)

if __name__ == "__main__":
    print("[START] Suspicious Behavior Detection Module Running...\n")

    while True:
        processes = load_process_data()

        if processes is not None:
            alerts = detect_suspicious_behavior(processes)
            export_alerts(alerts)
            display_alerts(alerts)
            print("\n-----\n")

            time.sleep(REFRESH_INTERVAL)

```

response.py

```

import os
import json
import time
import signal
import subprocess

```

```

from datetime import datetime

# Colors used to make terminal output easier to read
RED = "\033[91m"
GREEN = "\033[92m"
YELLOW = "\033[93m"
BLUE = "\033[94m"
RESET = "\033[0m"

# File names and time settings
ALERT_FILE = "alerts.json"
LOG_FILE = "response_logs.txt"
CHECK_INTERVAL = 2

# Stores how many times each PID has been flagged
offense_count = {}

# Processes that should not be changed or killed
SAFE_PROCESSES = [
    "systemd",
    "kthreadd",
    "lightdm",
    "Xorg",
    "dbus-daemon",
    "NetworkManager",
    "bash",
    "zsh",
    "sh",
    "python3",
    "qterminal",
    "xfce4-terminal",
    "gnome-terminal",
    "konsole",
    "xfdesktop",
    "xfwm4",
    "xfce4-panel",
    "xfsettingsd",
    "Thunar"
]

def write_log(message, color=RESET):
    # Add current date and time to each message
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    full_message = f"[{timestamp}] {message}"

    # Show message in terminal
    print(color + full_message + RESET)

    # Save the same message in the log file
    with open(LOG_FILE, "a") as file:

```

```

file.write(full_message + "\n")

def pid_exists(pid):
    # Check if a process with this PID still exists
    try:
        os.kill(pid, 0)
        return True
    except ProcessLookupError:
        return False
    except PermissionError:
        # Process exists, but access is limited
        return True
    except Exception:
        return False

def get_protected_pids():
    # Protect the current script, its parent, and related session processes
    protected = set()

    try:
        current_pid = os.getpid()
        parent_pid = os.getppid()

        protected.add(current_pid)
        protected.add(parent_pid)

        pid = current_pid
        while pid > 1:
            try:
                # Get the parent PID of the current PID
                ppid = int(subprocess.check_output(
                    ["ps", "-o", "ppid=", "-p", str(pid)],
                    text=True
                ).strip())
                protected.add(ppid)

                # Stop if the chain ends
                if ppid == pid or ppid <= 1:
                    break

                pid = ppid
            except Exception:
                break
        except Exception:
            pass

    return protected

def reduce_priority(pid):

```

```

# Lower the process priority using renice
try:
    subprocess.run(["renice", "+10", "-p", str(pid)], check=True)
    write_log(f"PID {pid}: priority reduced", GREEN)
except Exception as e:
    write_log(f"PID {pid}: failed to reduce priority -> {e}", RED)

def kill_process(pid):
    # Forcefully stop the process
    try:
        os.kill(pid, signal.SIGKILL)
        write_log(f"PID {pid}: process killed", RED)

        # Remove PID from offense tracking after killing it
        if pid in offense_count:
            del offense_count[pid]

    except Exception as e:
        write_log(f"PID {pid}: failed to kill -> {e}", RED)

def load_alerts():
    # Read alerts created by the detection module
    if not os.path.exists(ALERT_FILE):
        return []

    try:
        with open(ALERT_FILE, "r") as file:
            return json.load(file)
    except Exception as e:
        write_log(f"Could not read alerts file -> {e}", RED)
        return []

def clear_alerts():
    # Clear the alerts file after processing all alerts
    try:
        with open(ALERT_FILE, "w") as file:
            json.dump([], file)
    except Exception as e:
        write_log(f"Could not clear alerts file -> {e}", RED)

def respond_to_alert(alert):
    # Get alert details
    pid = alert.get("pid")
    violation = alert.get("violation")
    name = alert.get("name", "Unknown")

    if pid is None:
        return

```

```

# Get PIDs that belong to the current session
protected_pids = get_protected_pids()

# Skip trusted process names
if name in SAFE_PROCESSES:
    write_log(f"PID {pid} ({name}) is protected by name. Skipping action.", BLUE)
    return

# Skip the current terminal/session processes
if pid in protected_pids:
    write_log(f"PID {pid} ({name}) is part of current session. Skipping action.", BLUE)
    return

# Skip the process if it already ended
if not pid_exists(pid):
    write_log(f"PID {pid} ({name}) no longer exists. Skipping.", BLUE)
    if pid in offense_count:
        del offense_count[pid]
    return

# Update offense count for this PID
if pid not in offense_count:
    offense_count[pid] = 1
else:
    offense_count[pid] += 1

count = offense_count[pid]

# Log the alert and current offense number
write_log(
    f"ALERT: PID {pid} ({name}) detected for {violation} - offense #{count}",
    YELLOW
)

# First offense: only log
if count == 1:
    write_log(f"PID {pid}: first offense -> logging only", BLUE)

# Second offense: lower priority
elif count == 2:
    write_log(f"PID {pid}: second offense -> reducing priority", BLUE)
    reduce_priority(pid)

# Third or later offense: kill process
else:
    write_log(f"PID {pid}: repeated offense -> killing process", BLUE)
    kill_process(pid)

def response_controller():

```

```
# Start the response module
write_log("Response module started", BLUE)

while True:
    # Load alerts every few seconds
    alerts = load_alerts()

    if alerts:
        for alert in alerts:
            respond_to_alert(alert)

        # Clear alerts after handling them
        clear_alerts()

    time.sleep(CHECK_INTERVAL)

if __name__ == "__main__":
    response_controller()
```


Conclusion

We made a Secure Process Resource Monitoring and Limiting System for Linux as part of this project. The main goal of the system was to watch running processes, find suspicious resource use, and take action to protect the system from abuse. The project had three parts that worked together: process monitoring, finding suspicious behavior, and controlling the response.

This design made the system easy to understand, well-organized, and easier to test. The monitoring module got real-time data about running processes, like how much CPU and memory they were using, how many child processes they had, and how many files were open. The detection module then compared this information to set limits to find suspicious behavior, like using too much CPU or memory, possibly fork bomb activity, or having too many open files. When this kind of activity was found, alerts were made and sent to the response module. The response module dealt with the alerts in a step-by-step approach. For example, it logged the event, lowered the priority of the process, and stopped processes that kept causing problems when needed.

In general, the project successfully showed how monitoring, detection, and response can all work together to make operating system security better. It also helped us understand the importance of trusted process protection, suitable threshold values, and controlled response actions. This project gave us practical experience in Linux process management, teamwork, and building a simple security focused system.

References

- [1] Python Software Foundation, “`json` — JSON encoder and decoder,” *Python 3 Documentation*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [2] Python Software Foundation, “`time` — Time access and conversions,” *Python 3 Documentation*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [3] Python Software Foundation, “`os` — Miscellaneous operating system interfaces,” *Python 3 Documentation*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [4] Python Software Foundation, “`subprocess` — Subprocess management,” *Python 3 Documentation*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [5] G. Rodolà, “psutil documentation,” *psutil Documentation*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [6] M. Kerrisk, “`renice(1)` — Linux manual page,” *Linux man-pages*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [7] M. Kerrisk, “`kill(1)` — Linux manual page,” *Linux man-pages*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [8] M. Kerrisk, “`signal(7)` — Linux manual page,” *Linux man-pages*, 2026. [Online]. Accessed: Apr. 2, 2026.
- [9] Linux Kernel Documentation, “Control Group v2,” *The Linux Kernel Documentation*, 2026. [Online]. Accessed: Apr. 2, 2026.

If you want a shorter list, keep only [1]–[5] for the Python and monitoring part, and add [6]–[9] only if your report discusses response actions like `renice`, `kill`, or `cgroups`.

Plagism Report:

Blackboard

SafeAssign Originality Report

Secure Operating Systems- 61 • أنظمة التشغيل الامنة • Project report plagiarism check • Submitted on Mon, Apr 13, 2026, 12:23 PM

SALMA AHMED QASSE

View Report Summary

① University of Sharjah College of Computing and Informatics Department of Computer Engineering 1502370-Secure Operating Systems Project Title: Secure Process Resource Monitoring and Limiting System Project Group: U23102077 Aseel Khalid Al mabrouk U23106893 Salma Ahmed Qasse U23101050 Reem Mohammed sukkari Course Instructor: Dr. ② Khalid Javeed Spring 2025/2026 Acknowledgment We would like to express our sincere thanks to Dr. ③ Khalid Javeed for his valuable guidance and support throughout this project. Also, we would like to show gratitude towards Reem, Aseel, and Salma for their cooperation in this project, as it helped us complete all components of the system. This project was a valuable learning experience for all of us, and we are grateful for the opportunity to work together and apply our knowledge in a practical way. Abstract This project describes the design and implementation of a Secure Process Resource Monitoring and Limiting System for Linux. The primary motivation for this system is to detect and prevent the abuse of any process resource, which may affect the performance and stability of the computer system. The proposed system has been designed by developing three major modules: process monitoring, suspicious behavior detection, and response control. The Monitoring module collects dynamic information regarding the processes being executed on the system at a specific time, including the percentage of CPU utilization, memory usage, the number of child processes, and the number of open file

Submission UUID: 88b69d59-74b6-a5f0-c09e-7105003933c2 Attachment UUID: c2a3b969-418c-8d16-bf1d-9e29c94cc18f

Additional cont... 3 %

Report Secure OS.docx

Sources

INCLUDED SOURCES

Institutional database (4) 3 %

Global database (1) 0 %

Top sources